

Real-Time Mobile Money Fraud Detection for Rwanda

A Big Data pipeline with HDFS, Kafka, PySpark, MLlib, AWS RDS and Django

Course	Big Data Essentials
Lecturer	Dr. Kundan Kumar
Case study	Mobile money fraud detection in Rwanda
Date	October 2026

Group members

No.	Name	Student number
1	MUSABYIMANA Diane	101218
2	UGIZWENAYO Divine	101191
3	TUMUKUNDE Sandra	101483
4	TWAGIRAYEZU Emmanuel	101214
5	EMERIMANA Edwin Kennedy	101205
6	MANIRAGUHA Jean de Dieu	101189

Summary

This project builds a real-time fraud detection system for mobile money transactions in Rwanda with the tools taught in the course: HDFS for storage, Kafka for streaming, PySpark Structured Streaming for processing, Spark MLlib for prediction, AWS RDS (MySQL) for the results and Django for the dashboard.

Real mobile money data is confidential, so the group generated a synthetic dataset of 7.5 million transactions (1.06 GB) that follows how MTN MoMo and Airtel Money work across Rwanda's 30 districts. About 1.3% of the transactions are fraud, following six patterns such as SIM-swap takeover, night-time account draining and corrupt agents.

A Random Forest model was trained on 2024 and 2025 and tested on January to May 2026. The same months were then replayed through Kafka as a live stream. Spark scores each transaction in 5-second batches and saves it to AWS RDS, and a Django dashboard hosted on an AWS EC2 server shows the results live.

Table 1. Key results

Area	Result
Dataset	7,500,000 transactions, 1.06 GB, 1 January 2024 to 31 May 2026, 96,743 fraud rows (1.29%)
Storage	9 blocks of 128 MB in HDFS; the file system check reports HEALTHY
Model	Random Forest: AUC-ROC 0.91, AUC-PR 0.353, recall 60.1%, precision 29.8% on 1,281,702 unseen transactions
Streaming	Kafka producer reads from HDFS at 20 transactions per second; Spark saves about 100 scored transactions every 5 seconds
Live results	On 279,605 streamed transactions: 7,307 flagged (2.6%), precision 30.5%, recall 60.7%
Database	MySQL 8.4.9 on AWS RDS (Stockholm), encrypted at rest
Dashboard	Django page with a district map, live alert feed, charts and model results, refreshing every 5 seconds, at http://13.51.200.99

Contents

Summary.....	2
1. Introduction and objectives.....	4
2. Architecture and workflow.....	5
3. Technologies, versions and dependencies.....	6
4. Dataset and storage in HDFS.....	7
5. Model training with Spark MLlib.....	9
6. Database on AWS RDS.....	11
7. Kafka and Spark streaming.....	12
8. Django dashboard.....	15
9. Deployment on AWS EC2.....	17
10. Testing and findings.....	19
11. Challenges, limitations and conclusion.....	20
Appendix. How to run the system.....	21

1. Introduction and objectives

Mobile money is the everyday payment method for millions of people in Rwanda. Services such as MTN MoMo and Airtel Money are used to send money, pay bills, buy airtime and cash out at agents. Typical frauds are SIM-swap attacks, corrupt agents who help cash out stolen money, and rapid chains of transfers. They happen within minutes, so a report written the next day is too late and every transaction has to be checked as it happens.

The project builds an end-to-end Big Data system that does this. It follows the four objectives of the course guidelines:

- Real-time data processing: stream transactions with Kafka and process them with PySpark.
- Prediction modelling: train and compare models with Spark MLlib.
- Database integration: store the data and predictions in AWS RDS (MySQL).
- Web dashboard: show predictions and insights live in a Django application.

Real transaction data is confidential, so the dataset was generated by the group. It is designed to look like Rwandan mobile money, but the fraud patterns were defined by the group. The results therefore show that the pipeline works end to end, and they do not predict how the model would perform on real data.

2. Architecture and workflow

The system has two paths. The offline path trains the model once on the historical data in HDFS. The online path scores every new transaction as it arrives, using the saved model.

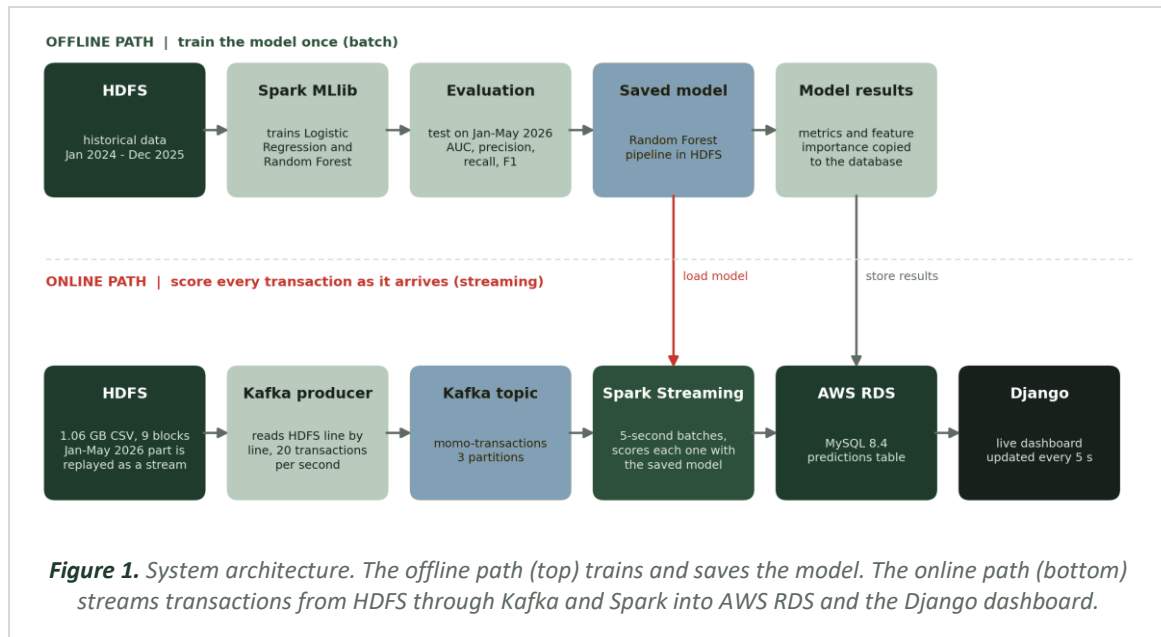


Table 2. Stages of the pipeline

#	Stage	Tool	What happens
1	Data creation	Python	Creates 7.5 million transactions in time order
2	Storage	HDFS 3.3.6	Holds the 1.06 GB file as 9 blocks
3	Model training	Spark MLlib	Trains two models on 2024 to 2025, tests on 2026, saves the best to HDFS
4	Database	AWS RDS MySQL	Creates the tables and stores the model results
5	Ingestion	Kafka producer	Reads HDFS and publishes transactions to a Kafka topic like live payments
6	Stream processing	Spark Structured Streaming	Reads the topic every 5 seconds, cleans the rows and predicts fraud
7	Result storage	AWS RDS MySQL	Saves every scored transaction with its fraud probability
8	Visualisation	Django 5.2	Reads RDS and shows alerts, a district map and charts
9	Deployment	AWS EC2 + gunicorn	Hosts the dashboard on a public server

3. Technologies, versions and dependencies

Everything ran on a Windows 10 laptop with 8 GB of RAM. Hadoop, Kafka and Spark run in Docker containers. The dashboard was built on the laptop and now runs on an AWS EC2 server.

Table 3. Main technologies and versions

Technology	Version	Used for
Hadoop (HDFS)	3.3.6	Distributed storage
Apache Kafka	3.7.0 (KRaft mode)	Message streaming
Apache Spark / PySpark	3.5.1	Training and Structured Streaming
Spark MLlib	3.5.1	Logistic Regression, Random Forest
Scala	2.12 (bundled with Spark)	Version of the connector JARs
Java	OpenJDK 17	Runs Spark in the container
Python	3.11 (Spark), 3.12.7 (laptop), 3.14.4 (server)	All scripts
MySQL on AWS RDS	8.4.9	Predictions and model results
Django	5.2.17	Dashboard
gunicorn	26.2.0	Production web server
Amazon EC2	t3.micro, Ubuntu Server 26.04 LTS	Hosts the dashboard
Docker Desktop	with Docker Compose	Runs HDFS, Kafka and Spark

Python libraries in the Spark container: pyspark 3.5.1, numpy 1.26.4, pandas 2.2.2, kafka-python 2.0.2, hdfs 2.7.3, pymysql 1.1.1 and cryptography 43.0.3. On the dashboard server: Django 5.2.17, PyMySQL 1.2.3, cryptography 50.0.2, python-dotenv 1.2.4, gunicorn 26.2.0, asgiref 3.12.1, sqlparse 0.6.0, cffi 2.1.1 and pycparser 3.0.

The Kafka connector is declared as `org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1`. The Scala version (2.12) and the Spark version (3.5.1) must match. Spark downloaded it with its dependencies: `spark-token-provider-kafka-0-10_2.12` 3.5.1, `kafka-clients` 3.4.1, `hadoop-client-runtime` and `hadoop-client-api` 3.3.4, `lz4-java` 1.8.0, `snappy-java` 1.1.10.3, `slf4j-api` 2.0.7, `commons-pool2` 2.11.1, `commons-logging` 1.1.3 and `jsr305` 3.0.0.

4. Dataset and storage in HDFS

`generate_dataset.py`, a plain Python script with a fixed random seed, creates the data in time order so that it can be replayed as a stream. Every account keeps a fixed profile, nights are quieter than days, and Kigali's districts are busier than rural ones.

Table 4. *The generated dataset*

Property	Value
Rows and size	7,500,000 transactions, 1,138,366,903 bytes (1.06 GB), 25 columns
Period	1 January 2024 to 31 May 2026
Accounts, agents, merchants	100,000 accounts, 5,000 agents (3% corrupt), 3,000 merchants
Geography and providers	30 districts in 5 provinces; MTN about 70%, Airtel about 30%
Transaction types	P2P transfer, cash out, cash in, merchant payment, airtime, bill payment, bank transfer
Fraud rows	96,743 (1.29%)

Table 5. *Fraud patterns in the dataset*

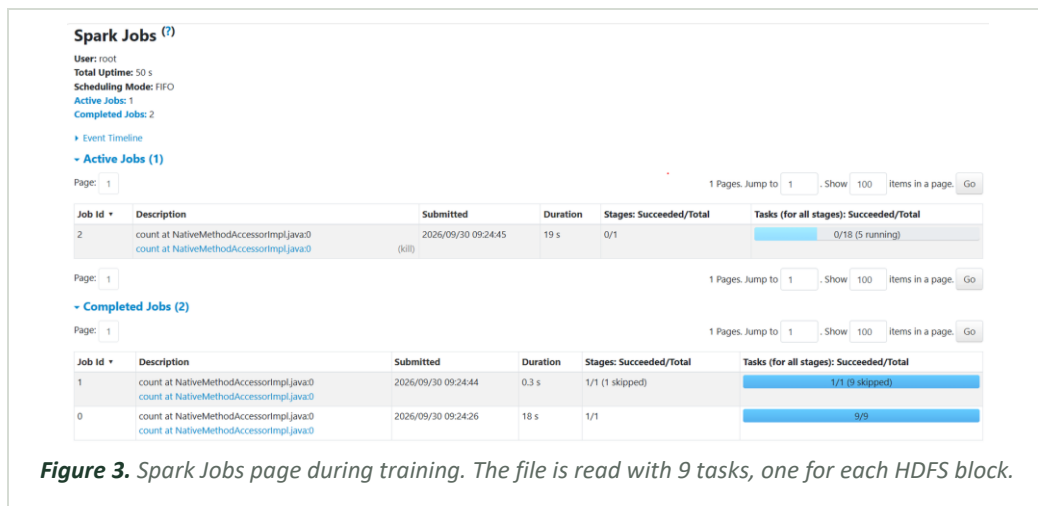
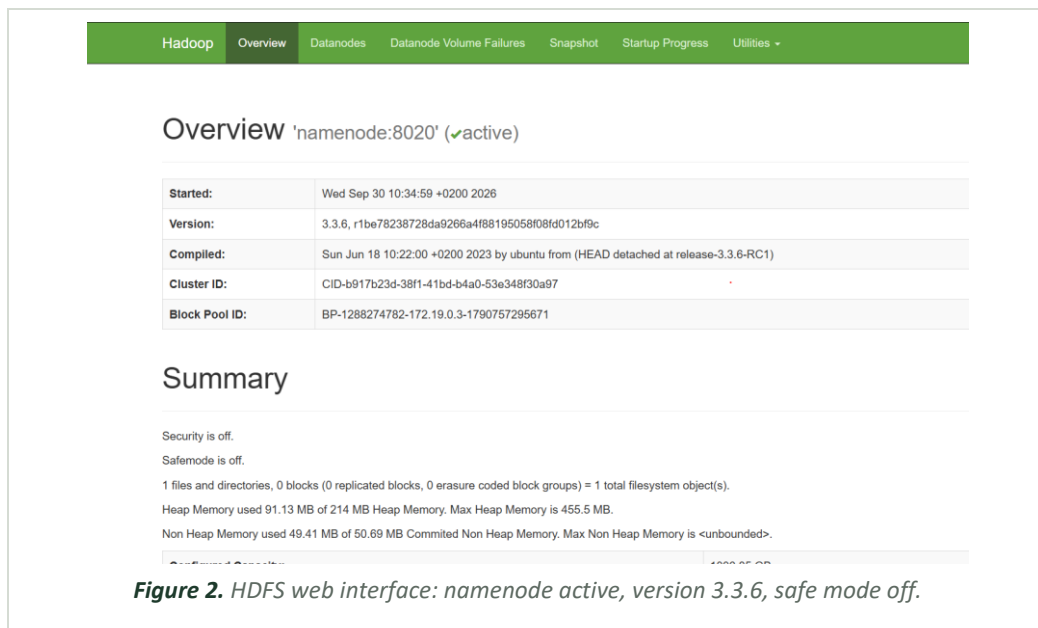
Pattern	When it applies	Chance
SIM-swap takeover	SIM swapped recently, new phone, money leaves the account	60%
Agent collusion	Cash out through a corrupt agent outside the customer's district	50%
Account drained at night	23:00 to 04:59, more than 90% of the balance leaves	30%
Large payment, new account	Account under 60 days old, amount above 80,000 RWF	30%
Burst of P2P transfers	5 or more transactions in the last hour	15%
Unusual amount spike	Amount more than 5 times the 7-day average	8%

A pattern does not always produce fraud, so the model has to learn patterns and cannot memorise rules. The column `fraud_type`, which names the pattern, is never given to the model because it contains the answer.

The file was uploaded to HDFS with `hdfs dfs -put`, and the file system check confirms it:

```
Status: HEALTHY
Total size: 1138366903 B
Total blocks (validated): 9 (avg. block size 126485211 B)
Missing blocks: 0
Corrupt blocks: 0
```

The file is stored as 9 blocks of up to 128 MB, so Spark reads it with 9 parallel tasks, one for each block.



5. Model training with Spark MLlib

`train_model.py` trains the model once on the historical data in HDFS:

1. Read the CSV from HDFS with a fixed schema and drop `fraud_type`.
2. Split by time: transactions before 1 January 2026 for training (694,307 balanced rows, 80,140 of them fraud) and January to May 2026 for testing (1,281,702 rows, 16,603 fraud).
3. Balance the training data by keeping all fraud rows and a 10% sample of the normal rows.
4. Build one pipeline with seven engineered features (account nearly emptied, amount against balance, amount against the 7-day average, night-time, money leaving, SIM swap on a new phone, new account), the encoded text columns and the original numeric columns.
5. Train Logistic Regression and a Random Forest (40 trees, depth 10), test both, and save the better one by AUC-PR to HDFS.

Table 6. Test results on 1,281,702 unseen transactions

Measure	Logistic Regression	Random Forest	Better
AUC-ROC	0.8895	0.9115	Random Forest
AUC-PR	0.2033	0.3533	Random Forest
Precision	22.1%	29.8%	Random Forest
Recall	53.3%	60.1%	Random Forest
F1-score	0.3120	0.3981	Random Forest

```

----- Random Forest -----
AUC-ROC   : 0.9115
AUC-PR    : 0.3533
Precision : 0.2976 (of transactions flagged as fraud, how many really were)
Recall    : 0.6013 (of all real frauds, how many we caught)
F1-score  : 0.3981
Accuracy  : 0.9765
Confusion matrix:
           predicted normal  predicted fraud
real normal      1,241,538      23,561
real fraud        6,620        9,983

```

Figure 4. Random Forest results on the test set, the model that was selected.

```

Training Logistic Regression...
  trained in 30s, evaluating on the test set...

---- Logistic Regression ----
AUC-ROC : 0.8895
AUC-PR  : 0.2033
Precision : 0.2206 (of transactions flagged as fraud, how many really were)
Recall    : 0.5326 (of all real frauds, how many we caught)
F1-score  : 0.3120
Accuracy : 0.9696
Confusion matrix:

```

	predicted normal	predicted fraud
real normal	1,233,867	31,232
real fraud	7,761	8,842

Figure 5. Logistic Regression results on the same test set.

- Accuracy is not a useful measure here. A system that always answered "not fraud" would score 98.70% accuracy while catching nothing, higher than the Random Forest's 97.65%.
- Of the transactions the Random Forest flags, 29.8% are real fraud, about 23 times the 1.30% base rate. It catches 1,141 more frauds than Logistic Regression and raises 7,671 fewer false alarms.
- It still misses about 40% of the fraud, and about 7 in 10 alerts are false. The balance can be moved by changing the decision threshold.

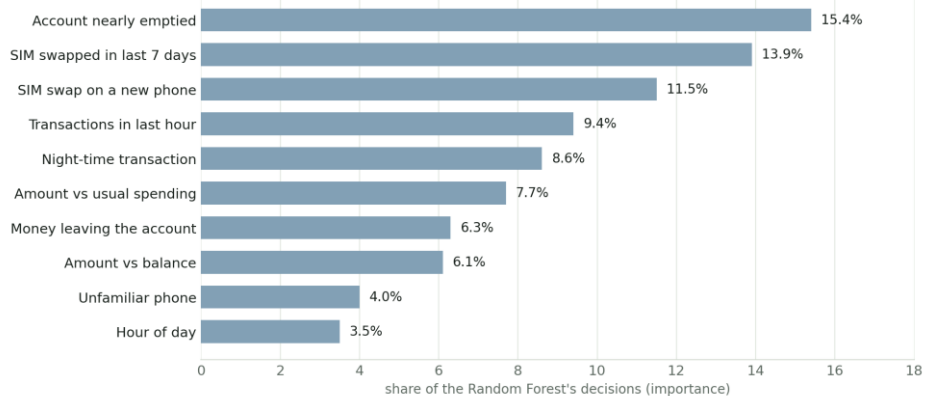


Figure 6. Feature importance in the Random Forest.

The strongest signals are an account being nearly emptied (15.4%), a recent SIM swap (13.9%), a SIM swap together with a new phone (11.5%), many transactions within an hour (9.4%) and night-time activity (8.6%). They agree with the patterns built into the data, so the model learned sensible behaviour and not noise.

6. Database on AWS RDS

Predictions and model results are stored in a managed MySQL database on Amazon RDS, which also keeps the database off the 8 GB laptop.

Table 7. RDS database settings

Setting	Value
Engine and name	MySQL Community 8.4.9, database momo_fraud
Region and size	Europe (Stockholm); db.t4g.micro with 20 GiB gp2 storage, autoscaling off
Access	Public access with a security group that allows port 3306 only from the developer's IP address and from the web server
Encryption	At rest (AWS KMS key aws/rds) and an encrypted connection
Login	Master user admin; the password is kept in a .env file, never in the code

setup_database.py creates three tables and loads the model results. predictions holds one row per scored transaction (with the fraud probability, the decision and the real label), model_metrics the scores of both models, and feature_importance the ten strongest features. transaction_id is the primary key, so with INSERT IGNORE a transaction that arrives twice is stored once.

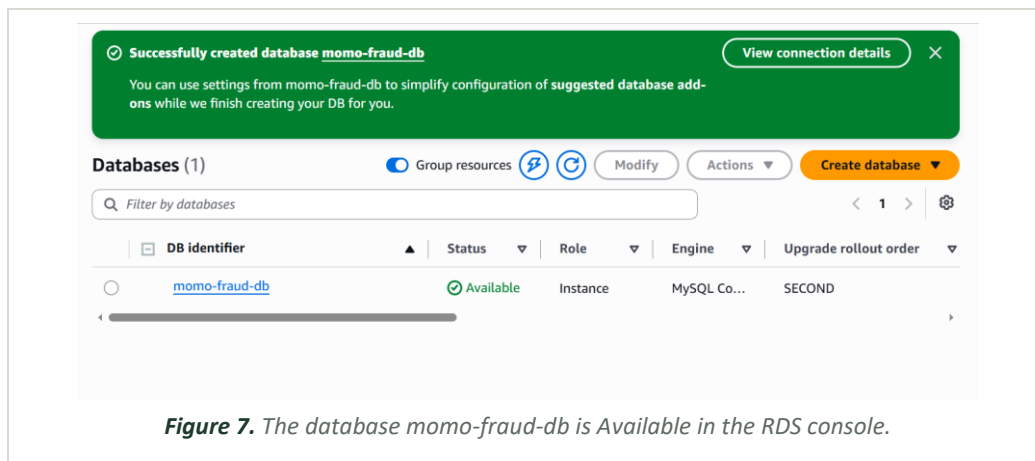


Figure 7. The database momo-fraud-db is Available in the RDS console.

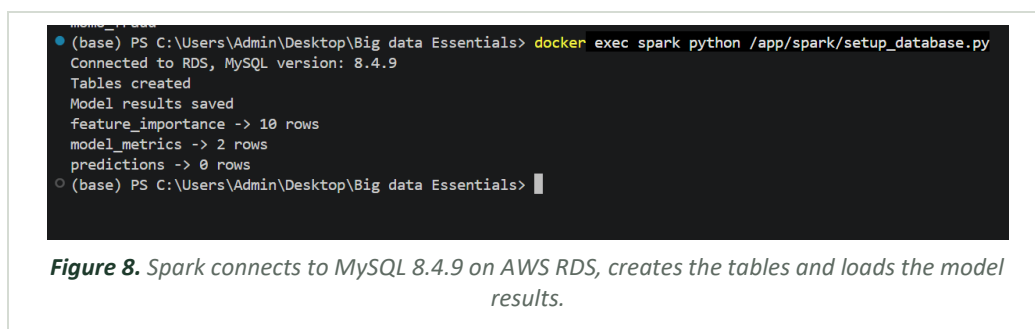


Figure 8. Spark connects to MySQL 8.4.9 on AWS RDS, creates the tables and loads the model results.

7. Kafka and Spark streaming

Kafka runs as one broker in KRaft mode. The topic `momo-transactions` has 3 partitions and replication factor 1. `producer.py` reads the transactions from HDFS line by line, skips everything before 2026 (the training period), and publishes them at 20 per second. The account id is the message key, so the transactions of one account stay in order in one partition. A bookmark file lets it continue after a restart without sending anything twice.

`streaming.py` is a Spark Structured Streaming job. Every 5 seconds it:

1. reads up to 500 messages from the topic and parses each CSV line into 25 columns;
2. cleans the batch with the rules below;
3. predicts fraud with the saved pipeline, which creates the features and returns the probability and the decision;
4. saves the rows to the predictions table in RDS;
5. writes a checkpoint, so that after a restart Spark continues where it stopped.

Table 8. *Cleaning rules*

Rule	Reason
Important fields must not be empty	A row without id, time, account, type, amount, balances or district cannot be scored
Amount above 0, balances not negative	Zero, negative or impossible values
Hour between 0 and 23, label 0 or 1	Catches broken times and labels
Type is one of the 7 known types	Rejects unknown types
Same transaction id twice in a batch kept once	Removes duplicates

The generated data is valid, so the real stream normally rejects nothing: batch 225: 30 received, 0 rejected by cleaning, 30 new saved to RDS, 1 flagged as fraud. During development a test batch with nine deliberately broken rows (a negative amount, a missing district, hour 27, a half-written line, a duplicate, an unknown type, a label of 2, a negative balance and text instead of a number) had all nine rejected, and none reached the database.

Restarting the producer from the beginning made Spark report "0 new" rows for the repeated transactions, and restarting Spark made the batch numbers continue where they stopped, as the screenshot shows.

```

:: retrieving :: org.apache.spark#spark-submit-parent-e321cd29-da04-4db8-8b3d-2
confs: [default]
0 artifacts copied, 11 already retrieved (0kB/42ms)
26/09/30 11:28:27 WARN NativeCodeLoader: Unable to load native-hadoop library for
. using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel
Model loaded from HDFS
Streaming started - waiting for transactions from Kafka...
batch 53: 95 new transactions saved to RDS, 1 flagged as fraud
batch 54: 376 new transactions saved to RDS, 5 flagged as fraud
batch 55: 59 new transactions saved to RDS, 0 flagged as fraud
batch 56: 123 new transactions saved to RDS, 1 flagged as fraud
batch 57: 111 new transactions saved to RDS, 0 flagged as fraud
batch 58: 123 new transactions saved to RDS, 1 flagged as fraud
batch 59: 104 new transactions saved to RDS, 0 flagged as fraud
batch 60: 106 new transactions saved to RDS, 2 flagged as fraud
batch 61: 117 new transactions saved to RDS, 3 flagged as fraud
batch 62: 88 new transactions saved to RDS, 2 flagged as fraud
batch 63: 119 new transactions saved to RDS, 6 flagged as fraud
batch 64: 99 new transactions saved to RDS, 1 flagged as fraud
batch 65: 101 new transactions saved to RDS, 3 flagged as fraud
batch 66: 108 new transactions saved to RDS, 1 flagged as fraud
batch 67: 88 new transactions saved to RDS, 1 flagged as fraud
batch 68: 86 new transactions saved to RDS, 0 flagged as fraud
batch 69: 86 new transactions saved to RDS, 1 flagged as fraud
batch 70: 89 new transactions saved to RDS, 2 flagged as fraud

```

Figure 9. After a restart, Spark continues at batch 53 and saves about 100 new transactions per batch.

```

(base) PS C:\Users\Admin\Desktop\Big data Essentials> docker exec -it spark python /app/producer/
producer.py
Reading /momo/raw/mobile_money_transactions.csv from HDFS
Skipping old transactions (takes 1-2 minutes)...
sent 100 transactions (last: MM0006218398 at 2026-01-01 00:45:12)
sent 200 transactions (last: MM0006218498 at 2026-01-01 01:38:27)
sent 300 transactions (last: MM0006218598 at 2026-01-01 02:31:25)
sent 400 transactions (last: MM0006218698 at 2026-01-01 03:25:59)
sent 500 transactions (last: MM0006218798 at 2026-01-01 04:20:00)
sent 600 transactions (last: MM0006218898 at 2026-01-01 05:27:40)
sent 700 transactions (last: MM0006218998 at 2026-01-01 06:05:03)
sent 800 transactions (last: MM0006219098 at 2026-01-01 06:18:57)
sent 900 transactions (last: MM0006219198 at 2026-01-01 06:32:07)
sent 1000 transactions (last: MM0006219298 at 2026-01-01 06:46:59)
sent 1100 transactions (last: MM0006219398 at 2026-01-01 07:01:09)
sent 1200 transactions (last: MM0006219498 at 2026-01-01 07:15:42)
sent 1300 transactions (last: MM0006219598 at 2026-01-01 07:30:31)
sent 1400 transactions (last: MM0006219698 at 2026-01-01 07:45:34)
sent 1500 transactions (last: MM0006219798 at 2026-01-01 08:01:12)
sent 1600 transactions (last: MM0006219898 at 2026-01-01 08:14:45)
sent 1700 transactions (last: MM0006219998 at 2026-01-01 08:28:00)
sent 1800 transactions (last: MM0006220098 at 2026-01-01 08:40:35)
sent 1900 transactions (last: MM0006220198 at 2026-01-01 08:55:21)
sent 2000 transactions (last: MM0006220298 at 2026-01-01 09:11:45)
sent 2100 transactions (last: MM0006220398 at 2026-01-01 09:24:13)
sent 2200 transactions (last: MM0006220498 at 2026-01-01 09:40:33)
sent 2300 transactions (last: MM0006220598 at 2026-01-01 09:55:06)
sent 2400 transactions (last: MM0006220698 at 2026-01-01 10:06:39)

```

Figure 10. The producer streaming January 2026 transactions from HDFS into Kafka.

```

(base) PS C:\Users\Admin\Desktop\Big data Essentials> docker exec kafka /opt/kafka/bin/kafka-cons
ole-consumer.sh --topic momo-transactions --bootstrap-server kafka:29092 --from-beginning --max-m
essages 5
MM0006218299,2026-01-01 00:00:27,ACC044736,MTN,57,2027,Burera,AIRTIME,USSD,1190,211109,209919,TEL
_MTN,Burera,Northern,1,0,0,3,0,1,1551,0,none
MM0006218300,2026-01-01 00:00:43,ACC008270,MTN,43,1122,Karongi,CASH_IN,AGENT,7080,44302,51382,AGT
01478,Karongi,Western,1,1,0,0,3,0,1,7512,0,none
MM0006218301,2026-01-01 00:00:54,ACC018086,MTN,37,3021,Gisagara,AIRTIME,USSD,1330,413368,412038,T
EL_MTN,Gisagara,Southern,1,0,0,0,3,0,2,870,0,none
MM0006218302,2026-01-01 00:01:10,ACC068627,MTN,49,1049,Ngoma,MERCHANT_PAYMENT,USSD,3790,228726,22
4936,MER01873,Gicumbi,Northern,0,0,0,0,3,0,6,3931,0,none
MM0006218303,2026-01-01 00:01:59,ACC059145,MTN,37,2220,Karongi,MERCHANT_PAYMENT,APP,550,28773,282
23,MER02565,Karongi,Western,1,0,0,0,3,0,2,398,0,none
Processed a total of 5 messages
(base) PS C:\Users\Admin\Desktop\Big data Essentials>

```

Figure 11. Reading five messages from the topic with the Kafka console consumer.

```

-----
|               | modules               | artifacts |
|   conf   | number | search | dwnlded | evicted | number | dwnlded |
|-----|-----|-----|-----|-----|-----|-----|
|   default |    11  |    11  |    11   |     0   |    11  |    11   |
|-----|-----|-----|-----|-----|-----|
:: retrieving :: org.apache.spark#spark-submit-parent-34aaff99-46f7-4ab1-9401-a5b5912728e3
confs: [default]
11 artifacts copied, 0 already retrieved (56767kB/294ms)
26/09/30 11:13:05 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform
. using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Model loaded from HDFS
Streaming started - waiting for transactions from Kafka...
batch 0: 498 new transactions saved to RDS, 50 flagged as fraud
batch 1: 498 new transactions saved to RDS, 11 flagged as fraud
batch 2: 499 new transactions saved to RDS, 6 flagged as fraud
batch 3: 498 new transactions saved to RDS, 11 flagged as fraud
batch 4: 499 new transactions saved to RDS, 6 flagged as fraud
batch 5: 498 new transactions saved to RDS, 8 flagged as fraud
batch 6: 86 new transactions saved to RDS, 1 flagged as fraud

```

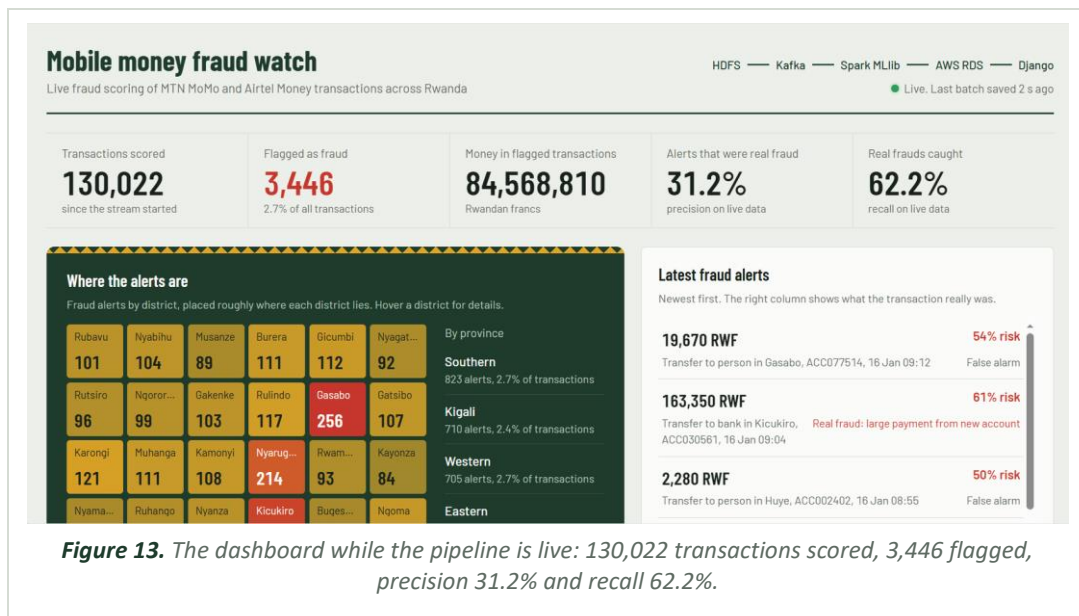
Figure 12. First streaming run: Spark loads the model and processes the backlog, then settles at about 100 transactions per batch.

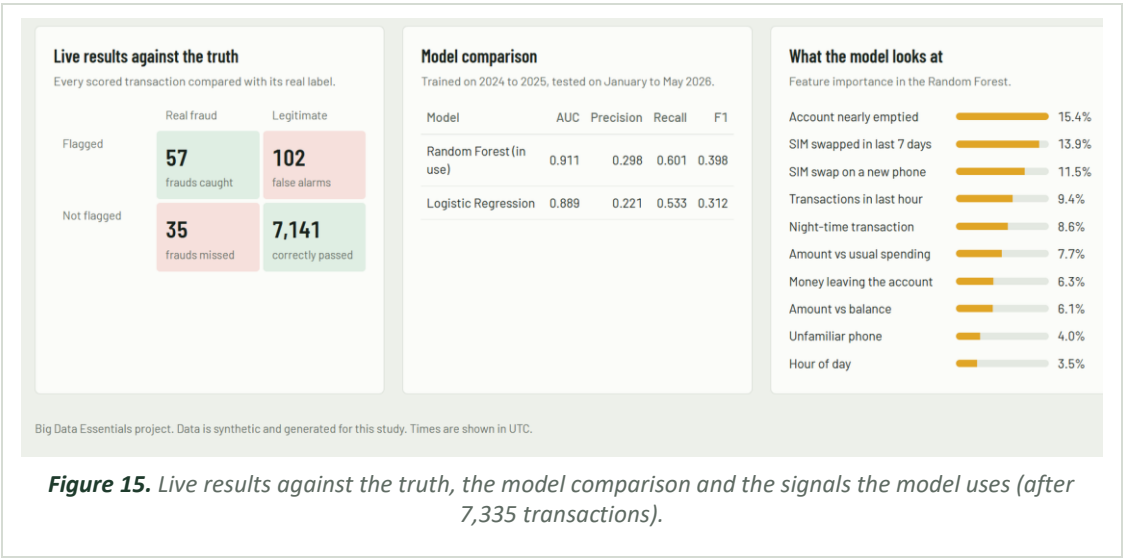
8. Django dashboard

The dashboard is one page of HTML, CSS and JavaScript, with Chart.js for the charts. Every 5 seconds it asks a Django endpoint (`/api/stats/`) for fresh numbers and redraws itself without reloading. `views.py` runs the queries through the Django ORM. The page shows "Live" with a green dot when Spark saved a batch in the last 30 seconds, and "Stream paused" otherwise.

Table 9. Dashboard panels

Panel	What it shows
Summary figures	Transactions scored, flagged, money in flagged payments, live precision and recall
District map	30 districts coloured by alerts, with a summary for each province
Latest fraud alerts	Newest alerts with amount, risk score and what the transaction really was
Stream activity	Transactions scored per minute and alerts raised
Alerts by hour and type	When and through which payment type fraud is flagged
Frauds caught by pattern	How many real frauds of each pattern were flagged
Live results	Frauds caught, false alarms, frauds missed and correct passes
Model and features	The two models side by side and the signals the Random Forest uses





9. Deployment on AWS EC2

The guidelines ask for the dashboard to be hosted so that users can open it. It runs on a small Amazon EC2 server in the same account and region as the database, so it works when the laptop is switched off. The public address is `http://13.51.200.99`. It uses plain HTTP because there is no domain name or certificate.

Table 10. EC2 server

Item	Value
Instance	momo-dashboard-server, t3.micro, Ubuntu Server 26.04 LTS, Python 3.14.4
Web server	gunicorn 26.2.0, 2 workers on port 80, started by a systemd service that restarts it if it stops
Code	Downloaded from GitHub (Diane-AUCA/Big-data-essentials_final-exam); the .env password file is not uploaded
Firewall of the server	Port 80 open to the internet; SSH through the browser-based EC2 Instance Connect
Firewall of the database	Port 3306 open only to the developer's IP address and to the server's security group

The code was prepared for the internet first: `settings.py` reads `DEBUG`, `ALLOWED_HOSTS` and `SECRET_KEY` from the `.env` file, the `/admin/` page was removed, and `requirements.txt` and `.gitignore` were added. On the server the commands were `git clone`, `python3 -m venv venv`, `pip install -r requirements.txt`, creation of the `.env` file with a hidden password prompt, and a `systemd` service for `gunicorn`. A test from the server counted 122,304 rows in RDS, and the service status shows active (running). Later changes need three steps: edit on GitHub, `git pull`, and `sudo systemctl restart momo-dashboard`.

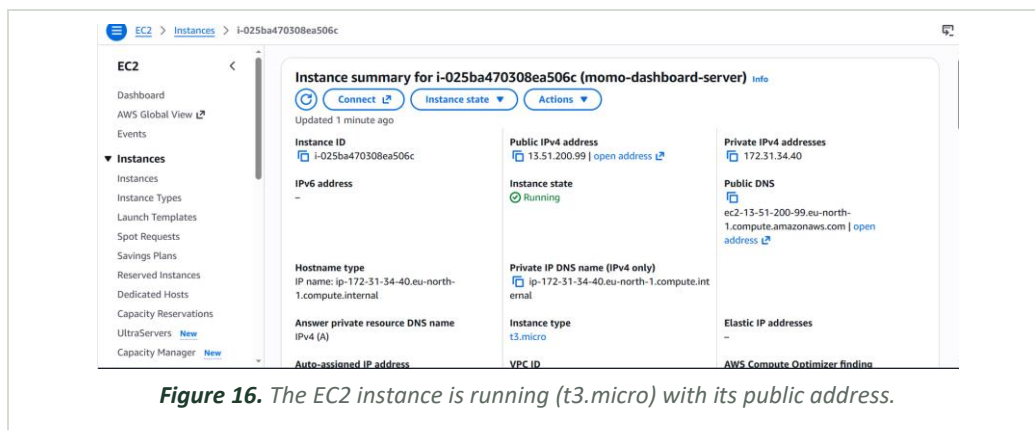
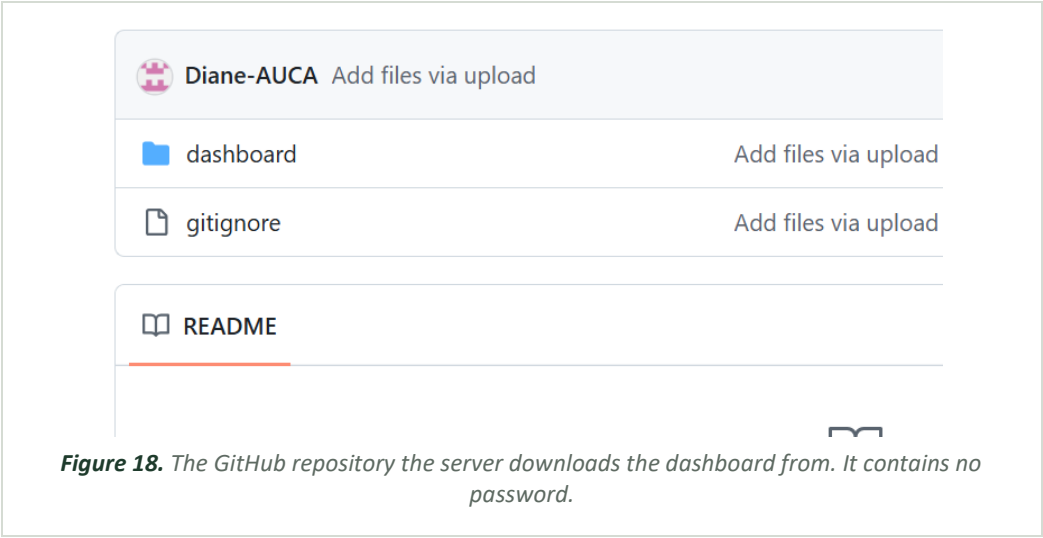
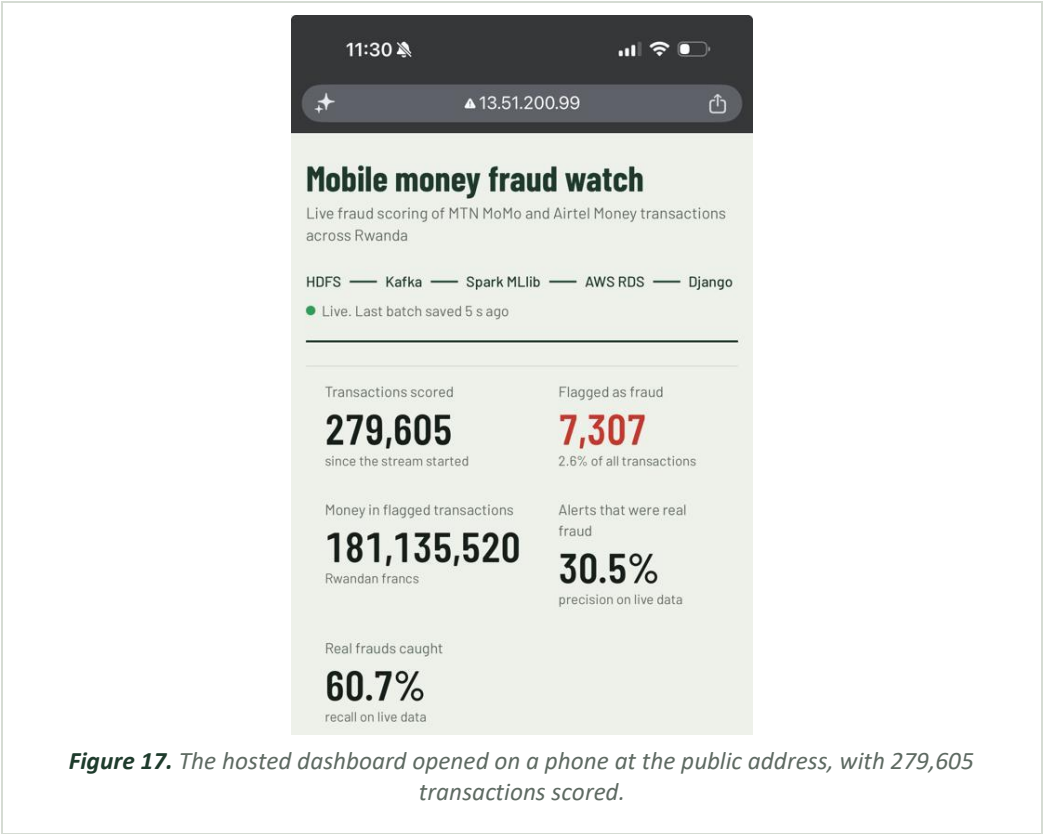


Figure 16. The EC2 instance is running (t3.micro) with its public address.



10. Testing and findings

Table 11. Checks and results

Check	How	Result
HDFS health	<code>hdfs fsck /momo/raw</code>	HEALTHY, 9 blocks, none missing or corrupt
Spark reads HDFS	Row count	7,500,000 rows
Kafka topic and messages	<code>kafka-topics --describe,</code> console consumer	3 partitions with leaders; messages received
Streaming speed	Streaming log	About 100 new transactions every 5 seconds
Duplicate protection	Producer restarted from the start	Spark saved 0 new rows
Restart recovery	Spark stopped and started	Batch numbers continued (53 after 52)
Data cleaning	Test batch with 9 broken rows	All 9 rejected, none stored
Database connection	<code>setup_database.py</code>	Connected to MySQL 8.4.9, three tables created
Hosted dashboard	systemd status, HTTP check, phone	Service running, page answered 200, opened on a phone

The Random Forest catches 60.1% of the fraud with a precision of 29.8% on transactions it never saw, and its flags are about 23 times more likely to be fraud than a random transaction. The live stream gives similar figures:

Table 12. Test results compared with the live stream

Measure	Test set (1,281,702)	Live (279,605)
Share of transactions flagged	2.6%	2.6%
Precision	29.8%	30.5%
Recall	60.1%	60.7%

The streamed transactions come from the same months that were kept for testing, so the live figures agree with the test results by construction. They show that the streaming pipeline scores like the offline test, not that the model was tested a second time. When 279,605 transactions had been scored, the model had flagged 7,307 of them, worth 181,135,520 RWF.

The model is strongest where the signals are visible: an emptied account, a SIM swap and night-time activity. It cannot find corrupt agents, because the agent is not an input, and a small share of the fraud is random by design. A provider could ask for an extra check after a SIM swap, limit payments that empty a wallet at night, review cash-outs far from the customer's home district and hold large payments from new accounts. The dataset is synthetic, so these findings show that the system works and are not a statement about real fraud levels.

11. Challenges, limitations and conclusion

Table 13. Problems and how they were solved

Problem	Cause	Solution
The Spark image would not build	The Python base image moved to a Debian release without Java 17	Pinned python:3.11-slim-bookworm
Only 8 GB of RAM	Hadoop, Kafka, Spark and a database do not all fit	One datanode, one broker, Spark in local mode, small limits, database on RDS
Only about 1% fraud	A rare-event problem	Balanced training set; AUC-PR, precision and recall instead of accuracy
Transactions arriving twice	Producer restarted from the beginning	Bookmark file, primary key with INSERT IGNORE, Spark checkpoint
Database timeouts	The RDS firewall allows one IP address, and home, office and hotspot differ	Updated the rule with "My IP"; the web server has its own rule
MySQL 8.4 login failed	The new password method needs an extra library	Added cryptography
Moving to the internet	Debug mode, key in the code, no production server	Settings from the environment, gunicorn, systemd, firewall rule between server and database

The main limitations are that the data is synthetic, that everything runs on one machine (one datanode, one broker, Spark in local mode), that labels are known immediately while real fraud labels arrive days later, that the agent is not a model input, and that the dashboard uses plain HTTP without a login. Future work includes an agent risk score, tuning the decision threshold, scheduled retraining, a domain name with HTTPS, a fixed IP address and a multi-node cluster.

The project built a complete real-time system for detecting mobile money fraud in Rwanda. Data stored in HDFS is replayed through Kafka, cleaned and scored by Spark with a Random Forest trained in MLlib, saved in MySQL on AWS RDS and shown on a Django dashboard hosted on AWS EC2. The model catches about 60% of the fraud with a precision of about 30%, and the pipeline kept working through restarts, repeated messages and changing network addresses. It covers each stage the course asked for and shows how the separate Big Data tools fit together in one system.

Appendix. How to run the system

Start Docker Desktop first and run the commands from the project folder in PowerShell.

Step	Command	What it does
1	<code>docker compose up -d</code>	Starts HDFS, Kafka and Spark
2	<code>docker exec namenode hdfs fsck /momo</code>	Checks that HDFS is healthy
3	<code>docker exec spark python /app/spark/train_model.py</code>	Trains and saves the model (first time only)
4	<code>docker exec spark python /app/spark/setup_database.py</code>	Creates the tables, tests the RDS connection
5	<code>docker exec -it spark python /app/spark/streaming.py</code>	Starts the streaming job
6	<code>docker exec -it spark python /app/producer/producer.py</code>	Starts the producer in a second terminal
7	Open <code>http://13.51.200.99</code>	The hosted dashboard
8	Ctrl+C in both terminals, then <code>docker compose stop</code>	Stops everything cleanly

If the database does not answer, a timeout usually means the internet connection has a new IP address. Open the security group `momo-db-sg` in the AWS console, edit the rule that holds the developer's address, choose "My IP" and save. The web server's rule stays unchanged. The dashboard code is on GitHub at https://github.com/Diane-AUCA/Big-data-essentials_final-exam.